

```
package library;

import java.sql.*;
import java.util.Scanner;
import java.util.regex.Pattern;

public class LibraryManagement {

    static final String URL = "jdbc:mysql://localhost:3306/library_db";
    static final String USER = "root";
    static final String PASS = "Shalu@1302";

    static Connection con;
    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            con = DriverManager.getConnection(URL, USER, PASS);

            System.out.println("Database connected successfully!");

            createTables();

            while (true) {
```

```
System.out.println("\n--- LIBRARY MANAGEMENT SYSTEM ---");
```

```
System.out.println("1. Add Book");
```

```
System.out.println("2. Search Book");
```

```
System.out.println("3. Add Member");
```

```
System.out.println("4. Issue Book");
```

```
System.out.println("5. Return Book");
```

```
System.out.println("6. Display Issued Books");
```

```
System.out.println("7. Update Book");
```

```
System.out.println("8. Delete Book");
```

```
System.out.println("9. Custom Query");
```

```
System.out.println("10. Exit");
```

```
System.out.print("Enter choice: ");
```

```
int ch = Integer.parseInt(sc.nextLine());
```

```
switch (ch) {
```

```
    case 1:
```

```
        addBook();
```

```
        break;
```

```
    case 2:
```

```
        searchBook();
```

```
        break;
```

```
    case 3:
```

```
        addMember();
```

break;

case 4:

issueBook();

break;

case 5:

returnBook();

break;

case 6:

displayIssues();

break;

case 7:

updateBook();

break;

case 8:

deleteBook();

break;

case 9:

customQuery();

break;

case 10:

System.**out**.println("Exiting...");

```

        return;

    default:
        System.out.println("Invalid choice!");
    }
}

} catch (Exception e) {
    e.printStackTrace();
}
}

// ===== CREATE TABLES =====

static void createTables() throws SQLException {

    Statement st = con.createStatement();

    st.execute(
        "CREATE TABLE IF NOT EXISTS books (" +
        "book_id INT AUTO_INCREMENT PRIMARY KEY," +
        "title VARCHAR(100) NOT NULL," +
        "author VARCHAR(100)," +
        "total_copies INT," +
        "available_copies INT)"
    );

    st.execute(

```

```

"CREATE TABLE IF NOT EXISTS members (" +
"member_id INT AUTO_INCREMENT PRIMARY KEY," +
"name VARCHAR(100) NOT NULL," +
"email VARCHAR(100) UNIQUE NOT NULL," +
"mobile VARCHAR(10))"
);

st.execute(
"CREATE TABLE IF NOT EXISTS book_issues (" +
"issue_id INT AUTO_INCREMENT PRIMARY KEY," +
"book_id INT," +
"member_id INT," +
"issue_date DATE," +
"return_date DATE," +
"status VARCHAR(20) DEFAULT 'ISSUED'," +
"fine INT DEFAULT 0," +
"FOREIGN KEY (book_id) REFERENCES books(book_id)," +
"FOREIGN KEY (member_id) REFERENCES members(member_id))"
);

System.out.println("Tables created successfully!");
}

// ===== EMAIL VALIDATION =====

static boolean isValidEmail(String email) {
return Pattern.matches(
"^[A-Za-z0-9+_.-]+@(.+)$",

```

```

        email

    );
}

// ===== ADD BOOK =====

static void addBook() throws SQLException {

    System.out.print("Enter Title: ");

    String title = sc.nextLine();

    System.out.print("Enter Author: ");

    String author = sc.nextLine();

    System.out.print("Enter Total Copies: ");

    int total = Integer.parseInt(sc.nextLine());

    if (total <= 0) {

        System.out.println("Copies must be greater than 0!");

        return;

    }

    PreparedStatement ps = con.prepareStatement(

        "INSERT INTO books(title, author, total_copies, available_copies) " +

        "VALUES(?,?,?,?)"

    );

    ps.setString(1, title);

```

```

ps.setString(2, author);

ps.setInt(3, total);

ps.setInt(4, total);


ps.executeUpdate();


System.out.println("Book added successfully!");
}


// ===== SEARCH BOOK =====


static void searchBook() throws SQLException {


    System.out.print("Enter Book Title to Search: ");
    String title = sc.nextLine();


    PreparedStatement ps = con.prepareStatement(
        "SELECT * FROM books WHERE title LIKE ?"
    );


    ps.setString(1, "%" + title + "%");


    ResultSet rs = ps.executeQuery();


    boolean found = false;


    while (rs.next()) {

```

```
found = true;
```

```
System.out.println(  
    "Book ID: " + rs.getInt("book_id") +  
    " | Title: " + rs.getString("title") +  
    " | Author: " + rs.getString("author") +  
    " | Total: " + rs.getInt("total_copies") +  
    " | Available: " + rs.getInt("available_copies")  
);  
}
```

```
if (!found) {  
    System.out.println("Book not found!");  
}  
}
```

```
// ===== ADD MEMBER =====
```

```
static void addMember() throws SQLException {
```

```
    System.out.print("Enter Name: ");
```

```
    String name = sc.nextLine();
```

```
    System.out.print("Enter Email: ");
```

```
    String email = sc.nextLine();
```

```
if (!isValidEmail(email)) {
```

```
    System.out.println("Invalid Email!");
```

```
    return;  
}
```

```
System.out.print("Enter Mobile (10 digits): ");  
String mobile = sc.nextLine();
```

```
if (!mobile.matches("[0-9]{10}")) {  
    System.out.println("Invalid Mobile!");  
    return;  
}
```

```
PreparedStatement ps = con.prepareStatement(  
    "INSERT INTO members(name, email, mobile) VALUES(?,?,?)"  
);
```

```
ps.setString(1, name);  
ps.setString(2, email);  
ps.setString(3, mobile);
```

```
ps.executeUpdate();
```

```
System.out.println("Member added successfully!");  
}
```

```
// ===== ISSUE BOOK =====
```

```
static void issueBook() throws SQLException {
```

```
System.out.print("Enter Book ID: ");  
  
int bookId = Integer.parseInt(sc.nextLine());
```

```
System.out.print("Enter Member ID: ");  
  
int memberId = Integer.parseInt(sc.nextLine());
```

```
con.setAutoCommit(false);
```

```
try {
```

```
    // Check book
```

```
    PreparedStatement chkBook = con.prepareStatement(  
        "SELECT available_copies FROM books WHERE book_id=?"  
    );
```

```
    chkBook.setInt(1, bookId);
```

```
    ResultSet rs = chkBook.executeQuery();
```

```
    if (!rs.next()) {  
        System.out.println("Book ID not found!");  
        con.rollback();  
        return;  
    }
```

```
    if (rs.getInt("available_copies") <= 0) {  
        System.out.println("Book not available!");  
        con.rollback();
```

```

    return;
}

// Check member
PreparedStatement chkMember = con.prepareStatement(
    "SELECT member_id FROM members WHERE member_id=?"
);

chkMember.setInt(1, memberId);

ResultSet memberRs = chkMember.executeQuery();

if (!memberRs.next()) {
    System.out.println("Member ID not found!");
    con.rollback();
    return;
}

// Insert issue
PreparedStatement ps = con.prepareStatement(
    "INSERT INTO book_issues(book_id, member_id, issue_date) " +
    "VALUES(?,?,CURDATE())"
);

ps.setInt(1, bookId);
ps.setInt(2, memberId);

ps.executeUpdate();

```

```

// Reduce available copy

PreparedStatement upd = con.prepareStatement(
    "UPDATE books SET available_copies = available_copies - 1 " +
    "WHERE book_id=?"
);

upd.setInt(1, bookId);

upd.executeUpdate();

con.commit();

System.out.println("Book Issued successfully!");

} catch (Exception e) {

    con.rollback();
    e.printStackTrace();

} finally {

    con.setAutoCommit(true);
}
}

// ===== FINE CALCULATION =====

```

```

static int calculateFine(int days) {

    int fine = 0;

    if (days > 14) {
        fine = (days - 14) * 5;
    }

    return fine;
}

// ===== RETURN BOOK =====

static void returnBook() throws SQLException {

    System.out.print("Enter Issue ID to Return: ");
    int issuelid = Integer.parseInt(sc.nextLine());

    con.setAutoCommit(false);

    try {

        PreparedStatement get = con.prepareStatement(
            "SELECT book_id, status, " +
            "DATEDIFF(CURDATE(), issue_date) AS days " +
            "FROM book_issues WHERE issue_id=?"
        );
    }

```

```
get.setInt(1, issueId);
```

```
ResultSet rs = get.executeQuery();
```

```
if (!rs.next()) {  
    System.out.println("Issue ID not found!");  
    con.rollback();  
    return;  
}
```

```
if (rs.getString("status").equals("RETURNED")) {  
    System.out.println("Already Returned!");  
    con.rollback();  
    return;  
}
```

```
int bookId = rs.getInt("book_id");
```

```
int days = rs.getInt("days");
```

```
// Fine calculation
```

```
int fine = calculateFine(days);
```

```
// Update issue
```

```
PreparedStatement upd1 = con.prepareStatement(  
    "UPDATE book_issues SET return_date=CURDATE(), " +  
    "status='RETURNED', fine=? WHERE issue_id=?"  
);
```

```
    upd1.setInt(1, fine);
    upd1.setInt(2, issued);

    upd1.executeUpdate();

    // Increase available copy
    PreparedStatement upd2 = con.prepareStatement(
        "UPDATE books SET available_copies = available_copies + 1 " +
        "WHERE book_id=?"
    );

    upd2.setInt(1, bookId);

    upd2.executeUpdate();

    con.commit();

    System.out.println(
        "Book Returned successfully!"
    );

    System.out.println(
        "Fine: Rs." + fine
    );

} catch (Exception e) {

    con.rollback();
```

```

        e.printStackTrace();

    } finally {

        con.setAutoCommit(true);
    }
}

// ===== DISPLAY ISSUED BOOKS =====

static void displayIssues() throws SQLException {

    ResultSet rs = con.createStatement().executeQuery(
        "SELECT bi.issue_id, b.title, m.name, " +
        "bi.issue_date, bi.return_date, bi.status, bi.fine " +
        "FROM book_issues bi " +
        "JOIN books b ON bi.book_id=b.book_id " +
        "JOIN members m ON bi.member_id=m.member_id"
    );

    boolean found = false;

    while (rs.next()) {

        found = true;

        System.out.println(
            "Issue ID: " + rs.getInt("issue_id") +

```

```

        " | Book: " + rs.getString("title") +
        " | Member: " + rs.getString("name") +
        " | Issue Date: " + rs.getDate("issue_date") +
        " | Return Date: " + rs.getDate("return_date") +
        " | Status: " + rs.getString("status") +
        " | Fine: Rs." + rs.getInt("fine")
    );
}

if (!found) {
    System.out.println("No issue records found!");
}
}

// ===== UPDATE BOOK =====

static void updateBook() throws SQLException {

    System.out.print("Enter Book ID to Update: ");

    int bookId = Integer.parseInt(sc.nextLine());

    PreparedStatement check = con.prepareStatement(
        "SELECT * FROM books WHERE book_id=?"
    );

    check.setInt(1, bookId);

    ResultSet rs = check.executeQuery();

```

```
if (!rs.next()) {  
    System.out.println("Book ID not found!");  
    return;  
}
```

```
System.out.print("Enter New Title: ");  
String title = sc.nextLine();
```

```
System.out.print("Enter New Author: ");  
String author = sc.nextLine();
```

```
System.out.print("Enter New Total Copies: ");  
int total = Integer.parseInt(sc.nextLine());
```

```
int issuedCopies =  
    rs.getInt("total_copies") -  
    rs.getInt("available_copies");
```

```
if (total < issuedCopies) {  
    System.out.println(  
        "Total copies cannot be less than currently issued copies!"  
    );  
    return;  
}
```

```
int available = total - issuedCopies;
```

```

PreparedStatement ps = con.prepareStatement(
    "UPDATE books SET title=?, author=?, " +
    "total_copies=?, available_copies=? " +
    "WHERE book_id=?"
);

ps.setString(1, title);
ps.setString(2, author);
ps.setInt(3, total);
ps.setInt(4, available);
ps.setInt(5, bookId);

ps.executeUpdate();

System.out.println("Book updated successfully!");
}

// ===== DELETE BOOK =====

static void deleteBook() throws SQLException {

    System.out.print("Enter Book ID to Delete: ");
    int bookId = Integer.parseInt(sc.nextLine());

    PreparedStatement checkBook = con.prepareStatement(
        "SELECT * FROM books WHERE book_id=?"
    );

```

```
checkBook.setInt(1, bookId);
```

```
ResultSet rs = checkBook.executeQuery();
```

```
if (!rs.next()) {
```

```
    System.out.println("Book ID not found!");
```

```
    return;
```

```
}
```

```
// Check currently issued
```

```
PreparedStatement checkIssue = con.prepareStatement(
```

```
    "SELECT COUNT(*) FROM book_issues " +
```

```
    "WHERE book_id=? AND status='ISSUED'"
```

```
);
```

```
checkIssue.setInt(1, bookId);
```

```
ResultSet issueRs = checkIssue.executeQuery();
```

```
issueRs.next();
```

```
if (issueRs.getInt(1) > 0) {
```

```
    System.out.println(
```

```
        "Cannot delete! Book is currently issued."
```

```
    );
```

```
    return;
```

```
}
```

```

// Delete old issue records

PreparedStatement deleteIssues = con.prepareStatement(
    "DELETE FROM book_issues WHERE book_id=?"
);

deleteIssues.setInt(1, bookId);
deleteIssues.executeUpdate();

// Delete book

PreparedStatement deleteBook = con.prepareStatement(
    "DELETE FROM books WHERE book_id=?"
);

deleteBook.setInt(1, bookId);

int rows = deleteBook.executeUpdate();

if (rows > 0) {
    System.out.println("Book deleted successfully!");
}
}

// ===== CUSTOM QUERY =====

static void customQuery() throws SQLException {

    ResultSet rs = con.createStatement().executeQuery(
        "SELECT m.name, COUNT(bi.issue_id) AS total " +

```

```

"FROM members m " +
"LEFT JOIN book_issues bi " +
"ON m.member_id=bi.member_id " +
"GROUP BY m.member_id"
);

boolean found = false;

while (rs.next()) {

    found = true;

    System.out.println(
        "Member: " + rs.getString("name") +
        " | Total Issued: " + rs.getInt("total")
    );
}

if (!found) {
    System.out.println("No members found!");
}
}
}

```